

JavaScript Debugging Breakpoints

[Back to JS page](#)

Table of Contents

- [JavaScript Debugging Breakpoints](#)
 - [Setting a Breakpoint](#)
 - [Example 1](#)
 - [The debugger Keyword](#)
 - [Example 2](#)
 - [Stepping Through Code](#)
 - [Example 3](#)
 - [The Scope Panel](#)
 - [Example 4](#)
 - [Watching Variables](#)
 - [Example 5](#)
 - [Common Beginner Mistakes](#)
 - [Example 6](#)
 - [When to Use Breakpoints](#)
 - [Example 7](#)
 - [Document](#)
 - [Reference](#)

Setting a Breakpoint

A breakpoint pauses code execution on a specific line. When the code stops, you can inspect variables and step through the code line by line. Breakpoints are set inside the browser developer tools. You do not guess values when using breakpoints—you see them.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Setting a Breakpoint</h4>
<p>Open the browser developer tools (F12), go to Sources tab, and click a line number to set a b
<p id="demo"></p>

<script>
function add(a, b) {
  let result = a + b;
  return result; // Set a breakpoint on this line
}

document.getElementById("demo").innerHTML = add(10, 5);
console.log("Result: " + add(10, 5));
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The debugger Keyword

The `debugger` keyword stops the execution of JavaScript and calls (if available) the debugging function. This has the same function as setting a breakpoint in the debugger. If no debugging is available, the `debugger` statement has no effect.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The debugger Keyword</h4>
<p>Open the browser console (F12). If DevTools is open, execution will pause at the debugger sta
<p id="demo"></p>

<script>
let x = 15 * 5;
debugger; // Execution pauses here if DevTools is open
document.getElementById("demo").innerHTML = x;

console.log("x = " + x);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Stepping Through Code

When execution is paused, you can control how the code runs. **Step Over** runs the next line. **Step Into** enters a function. **Step Out** exits the current function. Step slowly and watch how values change.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Stepping Through Code</h4>
<p>Open DevTools (F12), go to Sources tab. Set a breakpoint at the first line inside multiply(),
<p id="demo"></p>

<script>
function multiply(a, b) {
  let step1 = a * 2;    // Step Over: watch a*2
  let step2 = b * 3;    // Step Over: watch b*3
  let result = step1 + step2; // Step Over: watch final result
  return result;
}

let output = multiply(5, 4);
document.getElementById("demo").innerHTML = "Result: " + output;
console.log("multiply(5,4) = " + output);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The Scope Panel

The Scope panel shows which variables are available at the current line. It helps you understand where variables live. Local variables exist inside a function. Global variables exist everywhere.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Scope Panel</h4>
<p>Open DevTools (F12). Set a breakpoint inside the test() function to see local vs global scope
<p id="demo"></p>

<script>
let x = 10; // Global variable - visible in Scope panel everywhere

function test() {
  let y = 5; // Local variable - visible only inside this function
  console.log(x + y); // Set breakpoint here to see Scope panel
  return x + y;
}

document.getElementById("demo").innerHTML =
  "Global x = " + x + ", Function result = " + test();
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Watching Variables

The Watch panel lets you track variable values live. This is useful when values change many times. The value updates automatically as you step through the code. Watch variables instead of adding many `console.log()` calls.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Watching Variables</h4>
<p>Open DevTools (F12), go to Sources tab. Set a breakpoint at the start of the loop to watch co
<p id="demo"></p>

<script>
function calculateSum(count) {
  let total = 0;
  for (let i = 1; i <= count; i++) {
    total += i; // Set breakpoint here and watch 'i' and 'total'
    console.log("i =", i, "total =", total);
  }
  return total;
}

let result = calculateSum(5);
document.getElementById("demo").innerHTML =
  "Sum of 1 to 5 = " + result + " (check console to see each step)";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Common Beginner Mistakes

Beginners often forget to reload the page after adding a breakpoint. Breakpoints inside loops can trigger many times. If execution stops repeatedly, disable the breakpoint temporarily.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Common Beginner Mistakes</h4>
<p>Open DevTools (F12). Try setting a breakpoint inside the loop to see it trigger multiple times</p>
<p id="demo"></p>

<script>
let text = "";

// Mistake: Breakpoint inside loop triggers many times
for (let i = 0; i < 5; i++) {
  text += "Iteration " + i + "<br>"; // Set breakpoint here – triggers 5 times!
}

// Mistake: Variables change unexpectedly
let value = 10;
value = value * 2;
value = value + 5;
value = value - 3; // What is the value now? Don't guess – check the console!

console.log("Final value:", value);
text += "<br>Final value: " + value + " (check console)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

When to Use Breakpoints

Use breakpoints when values change unexpectedly, when code runs but produces wrong results, or when debugging complex logic. Breakpoints let you see exactly what's happening at every step of your code.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>When to Use Breakpoints</h4>
<p>Open DevTools (F12) and set a breakpoint to debug this calculation.</p>
<p id="demo"></p>

<script>
// This function has complex logic that could produce wrong results
function calculateDiscount(price, customerType) {
  let discount = 0;

  if (customerType === "VIP") {
    discount = 0.2;
  } else if (customerType === "Member") {
    discount = 0.1;
  } else {
    discount = 0.05;
  }

  let discountAmount = price * discount; // Set breakpoint to verify values
  let finalPrice = price - discountAmount;

  return finalPrice;
}

let price1 = calculateDiscount(1000, "VIP");
let price2 = calculateDiscount(500, "Member");
let price3 = calculateDiscount(300, "Guest");

console.log("VIP pays:", price1);
console.log("Member pays:", price2);
console.log("Guest pays:", price3);

document.getElementById("demo").innerHTML =
  "VIP: " + price1 + " (20% off 1000)<br>" +
  "Member: " + price2 + " (10% off 500)<br>" +
  "Guest: " + price3 + " (5% off 300)";
</script>

</body>
</html>
```


Example 7

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Debugging Breakpoints](#)